چالش ۳۰ روزهٔ اندروید

آمادگی مصاحبهٔ اندروید — جونیور ← مید ← سینیور ← آرکیتکت

نسخهٔ ۲۰۲۶

روز ۱۰

استراتژی‌های تست، بخش دوم — تست integration، UI، اسکرین‌شات و performance



اسماعیل مرادی

@esmaeeil-moradi700

چرا تست UI و integration در ۲۰۲۶ مهم است؟

unit testها (روز ۹) ثابت می‌کنند قطعه‌ها کار می‌کنند. این بخش ثابت می‌کند **کنار هم، روی صفحه و روی دستگاه واقعی** هم کار می‌کنند — بدون یک مجموعهٔ تست یک‌ساعته و flaky:

- Compose تست UI را **معنایی** کرد: nodeها را با معنایشان پیدا می‌کنید، نه با ID یک view
- Robolectric و تست اسکرین‌شات خیلی از تست‌های UI را **از دستگاه** به JVM آوردند
- رگرسیون‌های بصری را **diff اسکرین‌شات** می‌گیرد، نه کسی که در QA صفحه‌ها را یکی‌یکی باز می‌کند
- performance هم تست‌پذیر است: **Macrobenchmark** زمان startup و jank را با کد اندازه می‌گیرد
- Gradle Managed Devices اجرای تست روی دستگاه را تکرارپذیر می‌کند (اتصال به CI در روز ۲۳)

سطح	سؤال‌ها
جونیور	تست UI در Compose، کتابخانهٔ Espresso، همگام‌سازی تست‌ها
مید	تست یک صفحه، Robolectric، تست integration، بیرون رفتن از اپ
سینیور	تست اسکرین‌شات، تست‌های flaky، Macrobenchmark به‌عنوان تست
آرکیتکت	Gradle Managed Devices، sharding و ایزوله‌سازی، تست UI در مقیاس بزرگ



اسماعیل مرادی
@esmaeeil-moradi700

مدل ذهنی: هر تست کجا اجرا می‌شود

JVM	JVM + Robolectric	Device / GMD
logic, VMs, repositories	screens, DAOs, screenshots	E2E journeys, system UI, benchmarks
fastest	fast	slow, flakiest

- هر تست را در **ارزان‌ترین ستونی** بگذارید که هنوز می‌تواند باگ را بگیرد
- دستگاه برای چیزی است که فقط دستگاه ثابت می‌کند: رندر واقعی، UI سیستم، performance
- در همهٔ ستون‌ها تست‌ها **hermetic** هستند: بدون بک‌اند واقعی و بدون state مشترک بین تست‌ها

قاعدهٔ واحد پشت همهٔ جواب‌های امروز

تست UI فقط باید به یک دلیل fail شود: UI اشتباه است. نه شبکه، نه زمان‌بندی، نه تستی که قبل از آن اجرا شد.



اسماعیل مرادی
@esmaeeil-moradi700

جونیور

تست UI در Compose، کتابخانهٔ Espresso، همگام‌سازی تست‌ها



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. یک تست UI در Compose

```
@get:Rule val compose = createComposeRule()

@Test fun loginButton_enabledOnlyWithValidInput() {
    val vm = LoginViewModel(FakeAuthRepo())
    compose.setContent { AppTheme { LoginScreen(vm) } }
    val login = compose.onNodeWithText("Log in")
    login.assertIsNotEnabled()

    compose.onNodeWithTag("email").performTextInput("a@b.co")
    compose.onNodeWithTag("password").performTextInput("s3cret")

    login.assertIsEnabled()
}
```

- تست‌های Compose روی **درخت semantics** جست‌وجو می‌کنند — همان درختی که سرویس‌های دسترس‌پذیری می‌خوانند — نه روی viewها
- `createComposeRule()` محتوا را در یک activity خالی میزبانی می‌کند؛ `createAndroidComposeRule<MainActivity>()` یک activity واقعی را اجرا می‌کند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. پیدا کن، عمل کن، assert کن

مرحله	API
پیدا کردن	`onNodeWithText`، `onNodeWithTag`، `onNodeWithContentDescription`، `onNode(hasText("Pay") and hasClickAction())`
عمل	`performClick()`، `performTextInput()`، `performScrollTo()`، `performTouchInput { swipeUp() }`
assert	`assertIsDisplayed()`، `assertIsEnabled()`، `assertTextEquals()`، `assertDoesNotExist()`
دیباگ	`onRoot().printToLog("TAG")` درخت semantics را چاپ می‌کند

تلهٔ مصاحبه

«*تست UI در Compose به دستگاه نیاز دارد.*» — همان تست `createComposeRule()` با Robolectric روی JVM هم اجرا می‌شود (سؤال ۵). محل اجرا انتخابِ source set در Gradle است، نه یک API متفاوت.

نکتهٔ راه‌اندازی

`createComposeRule()` به `debugImplementation("androidx.compose.ui:ui-test-manifest")` نیاز دارد — این وابستگی activity خالی‌ای را که rule اجرا می‌کند در manifest تعریف می‌کند. بدون آن، rule نمی‌تواند شروع کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. Espresso برای صفحه‌های مبتنی بر View

```
@RunWith(AndroidJUnit4::class)
class CheckoutActivityTest {
    @get:Rule
    val rule = activityScenarioRule<CheckoutActivity>()

    @Test fun shortCardNumber_showsError() {
        onView(withId(R.id.card_number))
            .perform(typeText("4242"), closeSoftKeyboard())
        onView(withId(R.id.pay)).perform(click())

        onView(withText(R.string.error_card_invalid))
            .check(matches(isDisplayed()))
    }
}
```

- `onView(matcher)` پیدا می‌کند، `perform(action)` عمل می‌کند و `check(assertion)` بررسی می‌کند
- لیست‌ها: `RecyclerViewActions` از espresso-contrib؛ `onData()` فقط برای `AdapterView`
- صفحه‌های ترکیبی: APIهای Espresso و Compose با `createAndroidComposeRule` در یک تست کنار هم کار می‌کنند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. چیزهایی که مصاحبه‌گر چک می‌کند

تلهٔ مصاحبه

صبر با `Thread.sleep(2000)` «تا داده برسد». روی دستگاه سریع زیادی کند است و روی دستگاه کند زیادی کوتاه — منبع کلاسیک تست‌های flaky. به‌جایش روی شرط همگام شوید (سؤال ۳).

تلهٔ مصاحبه

«*حالا که Compose داریم، Espresso منسوخ شده.*» — بیشتر اپ‌های production هنوز صفحه‌های View دارند و Espresso هنوز ابزار آن‌هاست — از جمله Viewهایی که داخل Compose هستند و Compose داخل View.

سؤال تکمیلی

«`ActivityTestRule` یا `ActivityScenario`؟» — `ActivityTestRule` منسوخ شده است. `ActivityScenario` (یا `ActivityScenarioRule`) lifecycle را صریح کنترل می‌کند: `moveToState()`، `recreate()`، `onActivity { }`.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. تست‌ها چطور همگام می‌شوند — و چه چیزی را نمی‌بینند

	منتظر می‌ماند برای	منتظر نمی‌ماند برای
Espresso	خالی شدن صف پیام ترد اصلی؛ `IdlingResource`های ثبت‌شده	coroutineهای شما روی dispatcherهای پس‌زمینه؛ انیمیشن‌های در حال اجرا
rule تست Compose	recompositionهای معوق، layout و draw؛ انیمیشن‌های روی `mainClock`؛ idling resourceهای ثبت‌شده	کار پس‌زمینه‌ای که ثبت یا fake نشده

تلهٔ مصاحبه

«Espresso خودش منتظر انیمیشن‌ها و بارگذاری داده می‌ماند.» — فقط منتظر **ترد اصلی** و idling resourceهای ثبت‌شده می‌ماند. درخواستی روی `Dispatchers.IO` برایش نامرئی است و انیمیشن‌های سیستم تست را flaky می‌کنند — با `animationsDisabled = true` در `testOptions` خاموششان کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. درست صبر کردن

```
// Best: hermetic fakes that answer at once — no waiting

// Compose: wait for a condition, never for a duration
compose.waitUntil(timeoutMillis = 5_000) {
    compose.onAllNodesWithText("Paid")
        .fetchSemanticsNodes().isNotEmpty()
}

// Compose: drive animations by hand
compose.mainClock.autoAdvance = false
compose.onNodeWithText("Details").performClick()
compose.mainClock.advanceTimeBy(300)
compose.onNodeWithTag("details_panel").assertIsDisplayed()

// Espresso: expose real async work as an IdlingResource
IdlingRegistry.getInstance().register(idlingResource)
```

تلهٔ مصاحبه

پخش کردن `CountingIdlingResource.increment()` در کد production، آن را به تست‌ها گره می‌زند. fake یا dispatcher تست را inject کنید — آن‌وقت چیزی برای صبر کردن نمی‌ماند.



اسماعیل مرادی
@esmaeeil-moradi700

میدِ

تست یک صفحه، Robolectric، تست integration، بیرون رفتن از اپ



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. تست یک صفحه: state به داخل، event به بیرون

```
@Test fun errorState_showsMessage_andReportsRetry() {
    var retries = 0
    compose.setContent {
        AppTheme {
            FeedScreen(
                state = FeedUiState(userMessage = "Offline"),
                onRetry = { retries++ },
            )
        }
    }
    compose.onNodeWithText("Offline").assertIsDisplayed()
    compose.onNodeWithText("Retry").performClick()
    assertEquals(1, retries)
}
```

- **screen بی‌state** روز ۳ این‌جا جواب می‌دهد: هر state فقط یک `setContent` فاصله دارد — بدون ViewModel و بدون شبکه
- هر state را تست کنید: loading، خالی، خطا، محتوا، متن طولانی، فونت بزرگ



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. با semantics، nodeها را مثل کاربر پیدا کنید

- چیزی را ترجیح دهید که کاربر درک می‌کند: متن، content description، نقش. وقتی متن ثابتی نیست از `testTag` استفاده کنید.
- یک `Button` semantics فرزندانش را **ادغام** می‌کند — برای رسیدن به node فرزند `useUnmergedTree = true` را پاس بدهید.
- tagهای Compose همان resource ID نیستند. در ریشه `Modifier.semantics { testTagsAsResourceId = true }` را بگذارید تا UI Automator و Macrobenchmark بتوانند از `By.res("…")` استفاده کنند.
- بررسی خودکار دسترس‌پذیری: `AccessibilityChecks.enable()` در Espresso کتابخانهٔ Accessibility Test Framework را روی هر action اجرا می‌کند.

تلهٔ مصاحبه

«*Accessibility Scanner را در تست UI اجرا کنید.*» — Accessibility Scanner یک اپ **دستی** است. بررسی خودکار با Accessibility Test Framework یا همان ATF انجام می‌شود.

تلهٔ مصاحبه

رشته‌های انگلیسی هاردکدشده در assertها در زبان‌های دیگر می‌شکنند. آن‌ها را از resource بخوانید — با `compose.activity.getString(…)` در `createAndroidComposeRule` — یا از tag استفاده کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. Robolectric: تست اندروید روی JVM

```
// src/test — no emulator
@RunWith(AndroidJUnit4::class)
@Config(qualifiers = "w360dp-h640dp-xhdpi")
class SettingsScreenTest {
    @get:Rule val compose = createComposeRule()

    @Test fun darkModeSwitch_reportsChange() {
        var dark = false
        compose.setContent {
            SettingsScreen(dark = false, onDarkChange = {
                dark = it
            })
        }
        compose.onNodeWithTag("dark_switch").performClick()
        assertTrue(dark)
    }
}
```

- همان APIهای تست روی دستگاه: `AndroidJUnit4`، `ActivityScenario`، rule مربوط به Compose — runner تعیین می‌کند کجا اجرا شوند
- به `testOptions.unitTests.isIncludeAndroidResources = true` نیاز دارد



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. Robolectric در برابر دستگاه

	Robolectric	دستگاه یا امولاتور
سرعت	JVM، با forkهای موازی تست در Gradle	نصب، بوت، اجرا
دقت	کد واقعی فریم‌ورک؛ سرویس‌های سیستمی شبیه‌سازی‌شده	همه‌چیز واقعی
مناسب برای	صفحه‌ها، resourceها، DAOها، اسکرین‌شات با `@GraphicsMode(NATIVE)`	UI سیستم، رفتار خاص سازنده‌ها، رندر GPU، performance

تلهٔ مصاحبه

«*Robolectric اندروید را mock می‌کند، پس نتیجه‌ها ساختگی‌اند.*» — Robolectric **کد واقعی فریم‌ورک اندروید** را برای SDK تنظیم‌شده اجرا می‌کند و فقط بخش‌های native و سیستمی را با shadow جایگزین می‌کند. چیزی که نمی‌تواند ثابت کند — دستگاه واقعی، پروسه‌های دیگر، performance — در یک مجموعهٔ کوچک‌تر روی دستگاه می‌ماند.

سؤال تکمیلی

«*با Compose کار می‌کند؟*» — بله. نمونهٔ Now in Android گوگل تست‌های اسکرین‌شات Compose خودش را روی Robolectric اجرا می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. تست integration با Room واقعی و درون حافظه

```
@RunWith(AndroidJUnit4::class)
class ArticleDaoTest {
    private lateinit var db: AppDatabase

    @Before fun setUp() {
        db = Room.inMemoryDatabaseBuilder(
            ApplicationProvider.getApplicationContext(),
            AppDatabase::class.java,
        ).build()
    }

    @After fun tearDown() = db.close()

    @Test fun upsert_replacesExistingRow() = runTest {
        val dao = db.articleDao()
        dao.upsertAll(listOf(article(id = 1, title = "Old")))
        dao.upsertAll(listOf(article(id = 1, title = "New")))
        val rows = dao.observeAll().first()
        assertEquals("New", rows.single().title)
    }
}
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. مرزهای دیگر

مرز	تست integration
migrationهای Room	`MigrationTestHelper` همراه با schemaهای exportشده — برای همهٔ مسیرهای migration
شبکه	MockWebServer از OkHttp یا `MockEngine` در Ktor: کلاینت و serializer واقعی، پاسخ‌های آماده — روز ۱۱
گراف DI	`@TestInstallIn` و `@UninstallModules` در Hilt، bindingها را در کل اپ عوض می‌کنند — روز ۷
ناوبری	`TestNavHostController` — assert روی back stack

تلهٔ مصاحبه

«تست‌های دیتابیس درون‌حافظه‌ای ثابت می‌کنند migrationها درست‌اند.» — دیتابیس درون‌حافظه‌ای از روی schema **فعلی** تازه ساخته می‌شود؛ هیچ migrationی اجرا نمی‌شود. migrationها را با `MigrationTestHelper` تست کنید.

تلهٔ مصاحبه

تست integration روی بک‌اند واقعی staging: کند، با دادهٔ مشترک، و درست در بدترین لحظه از دسترس خارج. تست‌ها را hermetic نگه دارید و قرارداد API را جداگانه بسنجید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. بیرون رفتن از اپ: intent، مجوز، UI سیستم

```
@get:Rule val intents = IntentsRule()

@Test fun takePhoto_showsPreview() {
    val result = Instrumentation.ActivityResult(
        Activity.RESULT_OK, Intent(),
    )
    intending(hasAction(MediaStore.ACTION_IMAGE_CAPTURE))
        .respondWith(result)

    onView(withId(R.id.take_photo)).perform(click())
    onView(withId(R.id.preview)).check(matches(isDisplayed()))
}
```

- در Espresso-Intents، تابع `intended(…)` intentهای خروجی را verify می‌کند و `intending(…)` نتیجه را stub می‌کند — بدون نیاز به اپ دوربین
- وقتی دیالوگ مجوز موضوع تست نیست: `GrantPermissionRule.grant(Manifest.permission.CAMERA)`
- برای notificationها، دیالوگ مجوز، launcher و اپ‌های دیگر از UI Automator استفاده کنید: `UiDevice` و `By.text(…)`

تلهٔ مصاحبه

«*Espresso می‌تواند روی دیالوگ مجوز کلیک کند.*» — نمی‌تواند: دیالوگ متعلق به پروسهٔ دیگری است. Espresso فقط اپ شما را می‌بیند؛ از UI Automator استفاده کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سینیور

تست اسکرین‌شات، تست‌های flaky، Macrobenchmark به‌عنوان تست



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. تست اسکرین‌شات با Roborazzi

```
@RunWith(AndroidJUnit4::class)
@GraphicsMode(GraphicsMode.Mode.NATIVE)
@Config(qualifiers = RobolectricDeviceQualifiers.Pixel5)
class CheckoutScreenshotTest {
    @get:Rule val compose = createComposeRule()

    @Test fun checkout_errorState() {
        compose.setContent {
            AppTheme { CheckoutScreen(errorState, onPay = {}) }
        }
        compose.onRoot().captureRoboImage()
    }
}
```

```
./gradlew recordRoborazziDebug   # write golden images
./gradlew verifyRoborazziDebug   # fail on pixel differences
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. انتخاب ابزار اسکرین‌شات

ابزار	رندر با	نکته‌ها
Roborazzi	گرافیک native در Robolectric، روی JVM	بعد از تعامل هم می‌تواند عکس بگیرد؛ برای Compose و View
Paparazzi	layoutlib — موتور رندر preview در Android Studio — روی JVM	سریع؛ یک composable یا view را رندر می‌کند — بدون activity و بدون تعامل
Compose Preview Screenshot Testing	layoutlib، با استفادهٔ دوباره از `@Preview`ها	پلاگین Gradle گوگل؛ به‌صورت آزمایشی عرضه شد
اسکرین‌شات روی دستگاه	GPU واقعی	بیشترین دقت، کندترین، پرنویزترین

تلهٔ مصاحبه

ضبط تصویرهای golden روی Mac و verify روی CI لینوکسی: فونت و anti-aliasing فرق دارند، پس همه‌چیز «fail» می‌شود. ضبط و verify را در **یک محیط** انجام دهید و diff تصویرها را در pull request بررسی کنید.

تلهٔ مصاحبه

تست اسکرین‌شات رفتار را تست نمی‌کند — دکمه‌ای که پیکسل‌به‌پیکسل درست است ولی کاری نمی‌کند هم pass می‌شود. و بالا بردن آستانهٔ diff تا وقتی سبز شود، رگرسیون‌های واقعی را پنهان می‌کند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. تست‌های flaky: علت‌ها و راه‌حل‌ها

علت	راه‌حل
شبکه یا بک‌اند واقعی	fakeهای hermetic، MockWebServer
`sleep()` و حدس زدن زمان	شرط‌ها، idling resource، dispatcher تست
انیمیشن‌ها	`animationsDisabled = true`
نشت state بین تست‌ها (prefs، دیتابیس، singleton)	Test Orchestrator همراه با `clearPackageData`، دیتابیس درون‌حافظه‌ای تازه
وابستگی به ترتیب اجرا	هر تست همهٔ نیازهایش را خودش آماده کند
منطقهٔ زمانی، locale، تاریخ امروز	`Clock` و locale را inject کنید و در تست ثابت نگه دارید
امولاتورهای پربار	GMD با imageهای ATD، دستگاه‌های موازی کمتر

تست flaky تستی است که روی کد یکسان هم pass و هم fail می‌شود. همان چیزی را نابود می‌کند که مجموعهٔ تست برایش وجود دارد: اعتماد به build قرمز.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. ایزوله‌سازی و سیاست flakiness

```
android {
    defaultConfig {
        testInstrumentationRunner =
            "androidx.test.runner.AndroidJUnitRunner"
        testInstrumentationRunnerArguments["clearPackageData"] =
            "true"
    }
    testOptions {
        execution = "ANDROIDX_TEST_ORCHESTRATOR"
        animationsDisabled = true
    }
}
dependencies {
    androidTestUtil(libs.androidx.test.orchestrator)
}
```

- **Orchestrator**: هر تست در اجرای instrumentation جداگانهٔ خودش — یک crash فقط یک تست را fail می‌کند، نه کل اجرا را
- **سیاست:** شناسایی (نرخ flaky هر تست را ثبت کنید) ← قرنطینه با یک مالک مشخص ← رفع علت ریشه‌ای ← بازگرداندن

تلهٔ مصاحبه

«*تست‌های fail‌شده را سه بار retry کن.*» — retry قرمز را بدون رفع هیچ چیزی سبز می‌کند، و پشت بعضی از این تست‌های flaky یک race condition واقعی است که کاربرانتان هم به آن می‌خورند. retry کنید تا flakiness را **اندازه بگیرید**، نه تا پنهانش کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. تست performance با Macrobenchmark

```
@get:Rule val rule = MacrobenchmarkRule()

@Test fun coldStartup() = rule.measureRepeated(
    packageName = "com.example.app",
    metrics = listOf(StartupTimingMetric()),
    iterations = 10,
    startupMode = StartupMode.COLD,
) {
    pressHome()
    startActivityAndWait()
}

@Test fun feedScroll() = rule.measureRepeated(
    packageName = "com.example.app",
    metrics = listOf(FrameTimingMetric()),
    iterations = 5,
    setupBlock = { startActivityAndWait() },
) {
    device.findObject(By.res("feed")).fling(Direction.DOWN)
}
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. قواعد عددهای معنادار

- در یک ماژول جدا از نوع `com.android.test` قرار دارد و اپ را **از بیرون**، مثل کاربر، اجرا می‌کند
- یک build شبیه release را اندازه بگیرید: غیر debuggable، با R8، همراه با `<profileable android:shell="true"/>`
- روی **دستگاه فیزیکی** اجرا کنید — عددهای امولاتور نماینده نیستند و کتابخانه روی امولاتور خطا می‌دهد، مگر اینکه عمداً خاموشش کنید
- `CompilationMode.None()` را با حالت پیش‌فرض مقایسه کنید تا ببینید Baseline Profile چه چیزی به شما می‌دهد — روز ۱۸

تلهٔ مصاحبه

«*Macrobenchmark مخصوص Jetpack Compose است.*» — کل اپ را از بیرون اندازه می‌گیرد، چه View چه Compose. اندازه‌گیری یک تابع داخل پروسه کار **Microbenchmark** است.

تلهٔ مصاحبه

«*وقتی startup بدتر شود benchmark خودش fail می‌شود.*» — نمی‌شود؛ فقط نتیجه را به‌شکل یک فایل JSON با min، median و max بیرون می‌دهد. تشخیص رگرسیون یعنی مقایسهٔ اجراها در طول زمان در CI — روز ۲۸.



اسماعیل مرادی
@esmaeeil-moradi700

آرکیتکت

Gradle Managed Devices، sharding و ایزوله‌سازی، تست UI در مقیاس بزرگ



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. Gradle Managed Devices

```
android {
    testOptions {
        managedDevices {
            localDevices {
                create("pixel6api34") {
                    device = "Pixel 6"
                    apiLevel = 34
                    systemImageSource = "aosp-atd"
                }
            }
        }
    }
}
```

```
./gradlew pixel6api34DebugAndroidTest
```

- Gradle خودش image را دانلود می‌کند، امولاتور را می‌سازد، تست‌ها را اجرا و بعد خاموشش می‌کند — **همان دستگاه** روی هر لپ‌تاپ و هر runner در CI
- **گروه دستگاه‌ها** یک مجموعهٔ تست را در یک task روی گوشی، تبلت و قدیمی‌ترین API پشتیبانی‌شده اجرا می‌کند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. استفادهٔ درست از GMD

- **imageهای ATD** (`aosp-atd` و `google-atd`): اپ‌ها و سرویس‌هایی که تست‌ها به‌ندرت لازم دارند حذف شده‌اند — CPU و حافظهٔ کمتر برای هر امولاتور
- تستی که به چیزی وابسته است که ATD حذف کرده، روی image کامل `aosp` یا `google` اجرا شود
- پلاگین Firebase Test Lab برای GMD همین DSL را روی دستگاه‌های واقعی در cloud اجرا می‌کند

تلهٔ مصاحبه

«*امولاتور در CI برای همه‌چیز کافی است.*» — نه برای عددهای performance (سؤال ۱۰) و نه برای رفتار خاص سازنده‌ها. GMD اجرای امولاتور را **تکرارپذیر** می‌کند، نه نمایندهٔ همهٔ گوشی‌ها.

سؤال تکمیلی

«*این روی هر pull request چطور اجرا می‌شود؟*» — طراحی pipeline، cache و اینکه کدام مجموعه کی اجرا شود موضوع CI/CD است — روز ۲۳.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. sharding: تقسیم مجموعهٔ تست بین دستگاه‌ها

```
# AndroidJUnitRunner: run shard 0 of 4 on one device
adb shell am instrument -w -e numShards 4 -e shardIndex 0 \
  com.example.test/androidx.test.runner.AndroidJUnitRunner

# The same through Gradle
./gradlew connectedDebugAndroidTest \
  -Pandroid.testInstrumentationRunnerArguments.numShards=4 \
  -Pandroid.testInstrumentationRunnerArguments.shardIndex=0
```

```
# gradle.properties — GMD splits one suite over N emulators
android.experimental.androidTest.numManagedDeviceShards=4
```

تلهٔ مصاحبه

«*sharding مجموعهٔ کند و flaky را درست می‌کند.*» — فقط زمان کل را کم می‌کند. shardهایی که حساب کاربری یا دادهٔ بک‌اند مشترک دارند flakiness را بدتر می‌کنند. runner هم تست‌ها را بر اساس **تعداد** تقسیم می‌کند، نه مدت اجرا — یک کلاس کند می‌تواند یک shard را قفل کند؛ ابزارهایی مثل Flank بر اساس مدت اجرا متعادل‌تر تقسیم می‌کنند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. جعبه‌ابزار ایزوله‌سازی

تکنیک	چه مشکلی را حل می‌کند
sharding با `numShards`، shardهای GMD یا device farm	زمان کل اجرا
Test Orchestrator همراه با `clearPackageData`	crashی که کل اجرا را می‌کُشد؛ نشت state بین تست‌ها
بک‌اند hermetic از طریق DI	قطعی بک‌اند، تغییر دادهٔ تست
فیلتر اندازه: `@SmallTest`، `@LargeTest` و `-e size`	اجرای زیرمجموعهٔ درست در زمان درست
دادهٔ تست یکتا برای هر تست	برخورد shardها روی حساب‌های مشترک

مجموعهٔ تست سریع نتیجهٔ تست‌های ایزوله است — نه ماشین‌های بیشتر.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. تست UI در مقیاس بزرگ: الگوی robot

```
class CheckoutRobot(private val rule: ComposeContentTestRule) {
    fun enterCard(number: String) = apply {
        rule.onNodeWithTag("card").performTextInput(number)
    }
    fun pay() = apply {
        rule.onNodeWithText("Pay").performClick()
    }
    fun assertReceiptShown() = apply {
        rule.onNodeWithTag("receipt").assertIsDisplayed()
    }
}

fun checkout(
    rule: ComposeContentTestRule,
    block: CheckoutRobot.() -> Unit,
) {
    CheckoutRobot(rule).block()
}

@Test fun validCard_showsReceipt() {
    checkout(compose) {
        enterCard("4242 4242 4242 4242")
        pay()
        assertReceiptShown()
    }
}
```

- تست‌ها مثل **نیت کاربر** خوانده می‌شوند؛ وقتی UI عوض شود، یک robot را درست می‌کنید، نه چهل تست را



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. سبد تست UI

لایه	چه چیزی	کجا	چه تعداد
تست صفحه	همهٔ stateهای همهٔ صفحه‌ها	JVM با Robolectric	زیاد
تست اسکرین‌شات	صفحه‌های کلیدی در ترکیب theme، اندازهٔ فونت و RTL	JVM	زیاد و ارزان
integration	DAO، migration، شبکه، DI، ناوبری	JVM یا دستگاه	متوسط
سفرهای E2E	ورود، خرید، onboarding — با بک‌اند hermetic	GMD یا device farm	کم
benchmarkها	startup، اسکرول‌های کلیدی	دستگاه فیزیکی	انگشت‌شمار

تلهٔ مصاحبه

«*هر باگ production یک تست E2E می‌گیرد.*» — تست را در **پایین‌ترین سطحی** اضافه کنید که باگ را بازتولید می‌کند. E2E برای سفرهای کاربر است، نه برای هر رگرسیون.

جواب سینیور در یک جمله

«هر تست UI را به ارزان‌ترین محیطی ببر که هنوز باگ را می‌گیرد، دستگاه را برای چیزی نگه دار که فقط دستگاه ثابت می‌کند، و تست flaky را باگ بدان — نه نویز.»



اسماعیل مرادی
@esmaeeil-moradi700

روز ۱۰ — خلاصهٔ یک‌صفحه‌ای

- تست‌های Compose روی **درخت semantics** جست‌وجو می‌کنند: مثل کاربر پیدا کنید، `testTag` فقط به‌عنوان جایگزین.
- Espresso منتظر ترد اصلی و idling resourceها می‌ماند — **نه** coroutineهای پس‌زمینهٔ شما. انیمیشن‌ها را خاموش کنید.
- منتظر شرط بمانید با `waitUntil`؛ هرگز `sleep` نکنید.
- صفحه را با state به داخل و event به بیرون تست کنید — composableهای بی‌state با state جعلی.
- Robolectric فریم‌ورک واقعی را روی JVM اجرا می‌کند؛ دستگاه را برای بقیه نگه دارید.
- Room درون‌حافظه‌ای برای DAO، `MigrationTestHelper` برای migration، و MockWebServer برای شبکه.
- UI سیستم پروسهٔ دیگری است: UI Automator یا `GrantPermissionRule`، نه Espresso.
- تست اسکرین‌شات رگرسیون بصری را می‌گیرد — ضبط و verify در یک محیط.
- تست flaky: قرنطینه و رفع؛ retry اندازه می‌گیرد، درمان نمی‌کند. Orchestrator و `clearPackageData` ایزوله می‌کنند.
- Macrobenchmark روی دستگاه فیزیکی و build شبیه release؛ GMD به‌علاوهٔ sharding برای اجرای تکرارپذیر و سریع‌تر روی دستگاه.

روز ۱۱ در راه است: لایهٔ شبکه، بخش اول. این پست را ذخیره کنید و همراه ما باشید.



اسماعیل مرادی
@esmaeeil-moradi700